Automated Smoke Testing for Multi-ending Game Quests via Large Language Models

Anonymous Authors

***Abstract*—Modern game quests are increasingly trending toward high freedom and multiple endings, making their design and testing more complex. Smoke testing in games aims to rapidly validate quest functionality, but existing automated testing approaches face critical limitations when handling quests with branching narratives, and multiple possible endings. These challenges result in insufficient exploration of endings, redundant action sequences, and prohibitively high interaction costs. To address these challenges, we propose EndWeaver, an automated smoke testing framework specifically designed for multi-ending game tasks. EndWeaver leverages LLMs to interpret quest descriptions and game environments, generating high-level methods to guide global exploration, complemented by the generated primitive task sequences for fine-grained planning. By combining this multi-level planning structure with the dynamic construction of state-transition graphs, EndWeaver ensures comprehensive coverage of all possible endings and enables efficient testing. We evaluated EndWeaver on three game environments—Mini RPG, Overcooked Plus, and Minecraft—against baseline methods. In all environments, EndWeaver achieved stable quest completion with high reliability, significantly outperforming the baselines. In terms of testing cost, EndWeaver required only a few thousand steps, while baseline methods required millions of steps, all while ensuring comprehensive coverage of all possible quest endings. Ablation studies further demonstrated the critical contributions of each framework component, validating the effectiveness and robustness of our approach.**

***Index Terms*—Smoking Test, Large Language Model, Role-playing Game**

I. Introduction

In recent years, the gaming industry has expanded rapidly, with players demanding richer narrative freedom and greater content diversity [1]. Quests, as the core vehicle of player engagement, now frequently adopt multi-ending structures and highly flexible branching mechanisms [2]. While these features enhance player immersion, they also impose substantial complexity on quest design and verification. At the same time, live-service operations—characterized by seasonal updates, recurring events, and balance patches—have accelerated development cycles, with some studios releasing multiple internal builds per day and updating hundreds of quests weekly [3]. The combination of game complexity and frequent iteration cycles makes testing increasingly essential for maintaining both development velocity and consistent player experience quality.

As a built-in step in quest development, smoke testing [4] is used to verify whether newly implemented content can be executed properly [5]. Unlike traditional testing methods that aim for comprehensive bug detection, smoke testing focuses on providing early signals of executability—ensuring that newly created quests or levels can be played without critical issues [6]. This type of testing—verifying the correctness of core gameplay flows at an early development stage—is particularly critical in high-frequency development environments [7]. First, it provides game designers with rapid and explicit feedback to confirm the correctness of each iteration and avoid misunderstandings between game design and development teams [8]. Second, it prevents full-scale testing from proceeding when core functionality is broken, thereby avoiding the costly waste of testing resources [9]. However, as multi-ending quests become increasingly complex and updates occur more frequently, manual smoke testing faces significant challenges in terms of cost, consistency, and coverage—underscoring the need for automation.

Automated playtesting in games has been extensively studied. It initially emerged from model-based methodologies that aimed to simulate gameplay behavior through structured representations, including predefined scenario scripting [10], replaying recorded user interactions [11], and generating test cases from formal models such as UML statecharts [12] and Petri nets that describe game logic [13]. Subsequently, to better scale with the complexity of modern games, research has increasingly shifted toward agent-based methods, particularly those leveraging reinforcement learning (RL) agents capable of interacting with game environments for large-scale state-space exploration [14]–[16]. For example, Zheng et al. [17] combine deep reinforcement learning with evolutionary algorithms to balance "winning" (progressing through the game) and "exploration" (covering state space), thereby enhancing targeted bug discovery. More recently, to better reflect the diverse preferences of human players, some studies have proposed persona-centered approaches, in which agents simulate human play styles to make testing more representative of real gameplay. These include methods that employ procedurally generated personas [18] or player styles learned from expert demonstrations [19], [20], enabling test agents to capture a broader range of player behaviors.

However, the above approaches are not directly suited for smoke testing, which emphasizes rapid executability verification in multi-ending quests rather than exhaustive state coverage and bug discovery. Their utility, in the purpose of smoke testing, is limited by three key issues: (i) existing approaches often require millions or even billions of environment interactions to ensure convergence, and when encountering new quests, they usually need retraining or parameter adjustments, leading to prohibitively high interaction costs; (ii) these approaches, especially RL-based methods, usually rely on reward functions for optimization, which not only

suffer from sparse reward distributions that hinder effective learning, but also tend to converge to a single path in multi-ending quests, resulting in insufficient exploration; (iii) such approaches frequently generate test sequences with redundant or inefficient actions, making it difficult to produce compact and high-quality optimal paths.

To address these issues, we propose a novel automated smoke testing framework, named ENDWEAVER, with three core ideas: (i) leveraging large language models (LLMs) for their text understanding and broad knowledge to support reasoning over game quests; (ii) multi-level planning to ensure comprehensive exploration and efficient testing; and (iii) combining state-action transition graphs to eliminate redundant actions. For (i), ENDWEAVER leverages LLMs to interpret quest descriptions and game environments, thereby enabling semantic reasoning across diverse quests and reducing excessive environment interactions. For (ii), ENDWEAVER adopts a multi-level planning approach, decomposing quests into high-level abstract method and low-level specific primitive task, thereby enabling comprehensive exploration to cover all possible paths in multi-ending quests. For (iii), ENDWEAVER incorporates state-action transition graphs, which record action sequences and state transitions to eliminate redundant nodes and actions, thereby achieving the optimal paths.

The main contributions of this paper are as follows:

- We introduce a novel framework, ENDWEAVER, tailored for smoke testing multi-ending game quests. This framework leverages LLMs to understand and reason about the game quest, and utilizes hierarchical reasoning along with graph structure construction to enable comprehensive and efficient exploration of multiple endings.
- To address the unique challenges of testing multi-ending game quests, ENDWEAVER integrates three key components: (i) leveraging LLMs to interpret quest descriptions and reason about diverse quest structures, reducing excessive environment interactions; (ii) combining method generation with primitive task sequence planning to balance broad path exploration with precise quest execution; and (iii) designing a dynamic state-action transition graph to enable efficient path coverage, redundancy reduction, and critical path identification.
- We conduct comprehensive experimental evaluations, demonstrating the superior performance of our framework in terms of efficiency, path diversity, and computational cost compared to traditional baseline methods. Furthermore, we perform a thorough ablation study, which evaluates the contribution of each key component (i.e., LLM, multi-level planning, and the state-action transition graph).

The rest of the paper is organized as follows: Section II reviews related work on automated game testing and studies on LLMs in the gaming field. Section III defines the notation and problem for the smoke testing of multiple-ending quests. Section IV explains the technical details of the ENDWEAVER. Section V provides a comprehensive evaluation of the ENDWEAVER through three games. Finally, Section VI concludes the paper and explores potential directions for future research.

II. RELATED WORK

A. Game Testing

Game testing plays a critical role in the lifecycle of game software, aiming to ensure the stability, completeness, and overall user experience of the game before release and during subsequent updates [21]. Unlike conventional software applications, game software requires particular attention to gameplay mechanics, user interface, and performance aspects [22].

As the complexity of modern games continues to grow, traditional manual testing methods are increasingly challenged by the vast range of possible game states, actions, and narrative branches. In response, researchers and developers have explored various automated and AI-driven testing approaches. The following sections detail the evolution of these methods, starting with basic scripted automation, then moving on to AI-assisted techniques, and ultimately examining strategies designed to handle complex and dynamic testing scenarios.

Automation in Game Testing. For relatively simple and fixed game scenarios, automated testing is often carried out by scripting predefined actions [3], [23]. In this approach, testers simulate human player behaviors by designing possible actions and writing control scripts based on them. However, as game environments grow more complex, the cost and inefficiency of writing such custom scripts increase significantly. To enhance the level of automation in game testing, various artificial intelligence (AI) techniques have been applied in multiple areas, including GUI testing, the feasibility testing of game quest scripts, and functional testing [24].

AI-Driven Testing Methods. AI-driven approaches aim to reduce human involvement and improve testing efficiency. A common AI testing method is training agents to mimic human behavior to complete specified quests [20], [25], [26]. However, these methods depend heavily on the quality of human-generated experience data and are often constrained by issues related to generalization, limiting their applicability in different testing scenarios. To overcome this dependency on human experience data, some studies have proposed driving agents with predefined rewards to accomplish specific behaviors, thus achieving the testing objectives [27], [28]. Nevertheless, designing reward functions remains scenario-specific, and these reward-driven tests share similarities with imitation learning. Their effectiveness is limited because outcomes depend heavily on how rewards are designed. This makes it hard to achieve the comprehensive and flexible exploration required for thorough testing.

Testing Strategies in Complex Environments. In addition to simple goal-oriented testing approaches, another direction in automated game testing is training agents to explore game environments [29], attempting to detect errors in specific game states. Wuji [17] uses evolutionary algorithms to generate diverse agents, making it easier to explore new states and thus increasing the likelihood of discovering errors. Other works [30] explore the environment through agents with different preferences, but these preferences are defined by humans and do not change during the exploration process. Additionally, graph-based [31], [32] and tree-based methods [33],

[34] have been applied in game exploration. With the rise of quest descriptions in text format, text classification [35], [36] and other natural language processing methods [37], [38] can also be leveraged in game testing. Furthermore, in our previous study collaborated with NetEase [39], traditional language models (e.g., FastText, LTSM) have been explored to accelerate smoke testing by combining quest description texts with online planning. However, unlike LLMs, these traditional models have limited performance and are not suitable for complex quest descriptions and multi-ending quest scenarios.

B. Applications of LLMs in Games

Due to LLM's exceptional capabilities in natural language understanding, contextual reasoning, and inference, they are considered highly promising for enhancing and streamlining the design and development of games. Existing research has already integrated LLMs into numerous in-game applications, including acting as players [40], serving as NPCs [41], assisting players as assistants [42], providing game commentary [43], functioning as game masters in Tabletop RPGs [44], analyzing player data [45], designing level difficulty [46], and more recently even operating as game mechanics [47] through multimodal models that enable direct player interaction.

LLMs as Players in Games. Among the aforementioned applications of LLMs in games, the use most similar to this research is the use of LLMs as players in games, i.e., allowing LLMs to act as players who can make decisions and perform actions to complete specific quests within a game. A key challenge in enabling LLMs to "play" games lies in converting game states and actions into input formats that LLMs can process efficiently. For instance, GATO [48] leverages a multimodal dataset for training, directly processing visual inputs to achieve near-human performance in Atari games. However, this approach requires extensive, high-quality game video datasets, posing significant challenges in terms of cost and scalability.

As an alternative, some studies propose indirect interaction methods where LLMs do not directly engage with the game environment but instead rely on intermediary layers, such as predefined scripts or interpreters. This setting has been particularly applied in open-world games like Minecraft. For instance, JARVIS-1 [49] integrates multimodal models (MLMs) with LLMs to perceive the environment, plan actions, and execute complex quests in an open-world setting via predefined code layers. This enables efficient planning and robust quest execution in the Minecraft environment. Similarly, VOYAGER [40] utilizes GPT-4's code generation capabilities to interact with the Mineflayer API. By dynamically generating code snippets, VOYAGER can complete advanced instructions (e.g., "find the nearest tree") and continuously expand its skill library to improve quest efficiency. Plan4MC [50] introduces a hierarchical multi-quest framework that combines reinforcement learning with LLMs. By decomposing complex open-world quests into manageable sub-skills, Plan4MC significantly improves sample efficiency and quest completion rates. DECKARD [51] takes a similar approach, combining LLMs and RL to hypothesize and verify abstract world models (AWMs), enhancing exploration and sample efficiency in quests such as item crafting within Minecraft. In addition to acting as in-game players or assistants, LLMs have also been explored as strategic guides for reinforcement learning agents during the pretraining phase. For example, Du et al. [52] propose a novel framework (ELLM) where LLMs generate semantically meaningful, commonsense-aligned goals for RL agents to pursue. These goals, derived from natural language descriptions of environment states, serve as intrinsic rewards that steer the agent's exploration behavior toward task-relevant regions of the state space. Compared to traditional intrinsic motivation methods such as novelty-based exploration, this LLM-guided approach encourages more purposeful behaviors that are better aligned with downstream tasks. Compared with the above studies, our research differs in several key aspects. First, we approach the problem from a developer's perspective rather than that of a player, focusing on improving testing processes rather than simulating player behavior. Second, we incorporate a broader range of inputs beyond direct UI interactions, such as game metadata and development-side information, allowing our framework to leverage richer contextual details for more targeted and efficient testing. Third, our framework places a strong emphasis on path diversity and reliability, addressing the unique demands of multi-ending quests and ensuring that smoke testing captures the full complexity of branching narratives and gameplay possibilities.

III. Problem Formulation

This section first introduces the core terminology used in this paper to describe the game environment. Based on these core terms, it then provides a problem definition constructed on the Markov Decision Process (MDP).

A. Terminology and Notations

This paper targets the game that has Markovian characteristics, where the interaction between the agent and the game environment can be considered as a variation of MDP. To facilitate subsequent discussion, we outline below the terminology, concepts, and notations used to describe the interaction model between the intelligent agent and the game environment.

- S is the set of states, representing all possible states in the game, such as the player's position,various status and surrounding environment.
- A is the set of actions, representing all possible actions the intelligent testing agent can perform, such as moving, picking up items, and interacting with NPCs.
- T is the state transition function, $T : S \times A \rightarrow S$, which defines how the system transitions to a new state s_{t+1} when the system is in state s_t and the agent performs action a_t at time step t.
- Q is the quest descriptions, providing context and objectives for the tasks the intelligent agent needs to complete. These descriptions are often presented in natural language or other structured formats, guiding the player's decision-making process.

- E is the quest completion state set, representing the states where the quest is considered complete, such as achieving a certain game ending.
- I is the set of auxiliary information related to the game's development, such as source code, level design, and contextual data. This information is often represented in natural language or code and serves as a resource for the testing agent to guide its interactions within the environment.

At each time step t, the intelligent testing agent selects an action $a_t \in A$ based on the current state $s_t \in S$ and quest description Q. Upon performing the action, the system transitions to a new state $s_{t+1} = T(s_t, a_t)$. This process repeats iteratively until the agent reaches a quest completion state $s_t \in E$. The primary goal of the agent is to explore all possible state-action paths leading to a state within the quest completion set E, ensuring comprehensive exploration and validation of the quest execution. Formally, this quest verification process can be framed as a state-action path search problem, where the agent begins at the initial state $s_0 \in S$, selects appropriate actions based on the quest description, and transitions through successive states until a quest completion state $s_t \in E$ is reached.

B. Problem Definition

While the MDP formulation provides a fundamental model of states and actions, complex quests often require a structured approach to bridge from abstract objectives to concrete executable actions. Drawing inspiration from Hierarchical Task Network (HTN) planning, we introduce a hierarchical refinement process, which not only refines the quest description Q step-by-step but also leverages the auxiliary information I to guide each layer of refinement. This ensures that the derived tasks and actions remain logically consistent with both the quest goals and the underlying game design.

Definition 1 (Game-Agent Interaction): The game environment is defined as a 6-tuple $M = \langle S, A, T, Q, E, I \rangle$, where S is a set of states, A is a set of actions, T is a state transition function, Q is descriptions, E is a quest completion state set, and I is a set of auxiliary information related to the game. The intelligent agent interacts with the game environment based on a hierarchical design. The hierarchical design consists of three levels from top to bottom: method $\hat{\hat{\pi}}$, primitive task $\hat{\pi}$, and action-level execution π. method is defined as $\hat{\hat{\pi}} : Q \times I \rightarrow \mathcal{G}$, where $\mathcal{G}$ is a set of abstract quest goals, and each abstract quest goal $g_h \in \mathcal{G}$ can correspond to one or more trajectories from the starting state to the completion state. primitive task is defined as $\hat{\pi} : \mathcal{G} \rightarrow \mathcal{X}$, where $\mathcal{X}$ is a set of pattern sequences. A pattern $g_l \in \mathcal{G}_{low}$ corresponds to a sequence of actions with clear semantics in the game, $\mathcal{G}_{low}$ is the space of the pattern. Action-level execution is defined as $\pi : \mathcal{G}_{low} \times S \rightarrow A$. $\square$

Through this hierarchical refinement, the agent does not need to reason over the entire action space directly. Instead, it begins with high-level quest information, step-by-step refines that information into actionable strategies and tasks, and ultimately arrives at fully specified game actions. This approach both simplifies the planning process and ensures that the testing agent can systematically explore and validate complex, multi-ending quests while maintaining coherence with the underlying game design.

IV. EndWeaver

In this section, we first introduce the overview EndWeaver, followed by a detailed explanation of each core component of the framework. Finally, we provide a concise description of the design of the LLM prompts.

A. Overview of EndWeaver

To address the challenges posed by smoke testing multi-path and multi-ending quests in modern games, we propose a hierarchical planning framework, EndWeaver, that employs LLMs for multi-level planning. Specifically, we decompose the quest planning process into four hierarchical layers—root task, method, primitive task, and input action—to ensure efficient and diverse path exploration while maintaining precise task execution. By further integrating the state-action transition graph, we maximize path coverage and effectively eliminate redundant routes.

The main steps of the framework are illustrated in Fig. 1:

1) **Method Generation**: At this stage, the goal is to break down a complex, multi-ending quest into a set of high-level method strategies that serve as abstract solutions for fulfilling the quest. For example, in a kitchen-themed game scenario where the quest is to submit a tomato-containing dish, the framework can derive multiple method from the quest information and game design details—such as "prepare an onion-tomato salad" or "prepare a lettuce-tomato salad." By doing so, subsequent planning processes can consider various approaches to completing the quest rather than relying on a single fixed pathway.
2) **Primitive Task Generation and Input Action Mapping**: Once a method is determined, the next step involves decomposing it into smaller, more reusable primitive task. Continuing the previous example, to achieve the method "prepare an onion-tomato salad," the system might generate primitive task such as "pick up a tomato" or "chop the onion." A function generated by the LLM then maps each primitive task to the corresponding game input actions (e.g., pressing a sequence of specific keys), enabling the framework to execute these tasks within the game environment.
3) **Primitive Task Sequence Assembly**: In this step, multiple primitive task are combined into consecutive sequences to fulfill the selected method and ultimately achieve the game goal. For instance, to complete the onion-tomato salad, the framework might produce a sequence like: "pick up a tomato," then "chop the tomato," next "pick up an onion," then "chop the onion," and finally "assemble the ingredients on a plate." Multiple such sequences may be generated to cover different route of reaching the same goal, ensuring comprehensive test coverage.

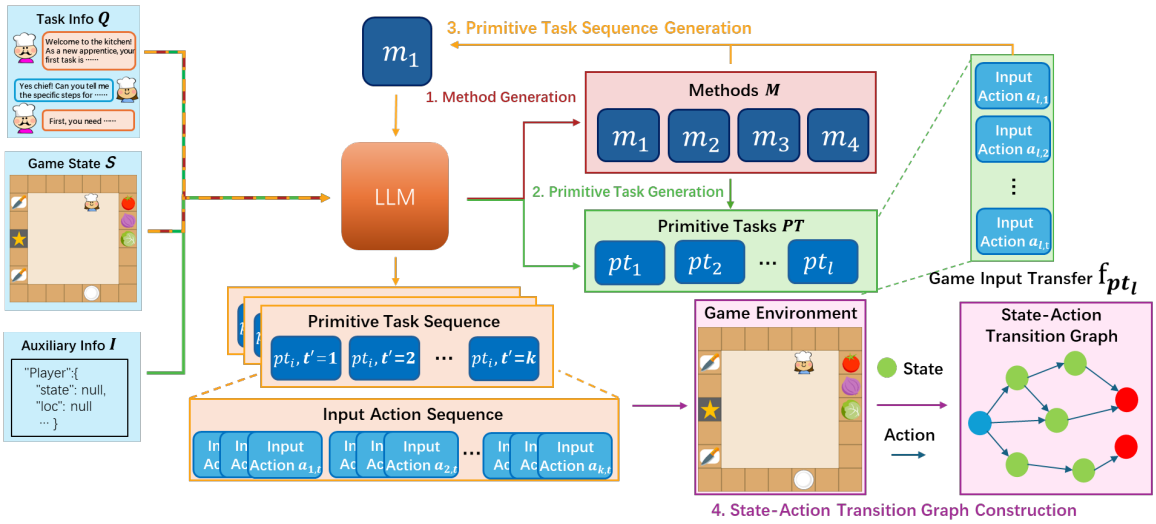


Fig. 1: The overview of ENDWEAVER.

4) **State-Action Transition Graph Construction**: As each primitive task sequence is executed, the system records the corresponding game states and their transitions, ultimately constructing a graph that captures all explored paths. In the salad-making scenario, each input action required by a primitive task (e.g., the actions needed to pick up an ingredient) leads to a new game state. By tracking these states and their triggering actions, the system builds a state-action transition graph. This facilitates systematic path analysis and metric extraction while eliminating redundant routes, thus supporting targeted testing of critical branching scenarios in multi-ending quests.

B. Step1: Method Generation

In the ENDWEAVER framework, we treat multi-ending quests as compound tasks, which are inherently complex goals composed of multiple interlinked subgoals and branching solution paths. To effectively address these complexities, we first introduce methods as high-level strategies guiding the pursuit of the overarching quest goal.

The first stage of our framework focuses on generating methods by analyzing quest descriptions and auxiliary information. Utilizing the natural language understanding and reasoning capabilities of LLMs, we decompose the compound quest into abstract methods. These methods represent diverse pathways to quest completion and provide a foundation for subsequent lower-level planning.

The method generation process can be modeled as mapping quest descriptions and auxiliary information to a set of abstract methods. Let Q represent the quest descriptions, I represent auxiliary information (e.g., game design information), and P represent the prompt designed to guide the LLM in generating objectives. Then, the method generation process can be expressed as:

$$(Q, I, P) \xrightarrow{\text{LLM}} M = \{m_h \mid h = 1, \ldots, n\}$$

where $M = \{m_h\}$ is a set of methods generated by the LLM. Each $m_h \in M$ represents a possible method to complete the quest. Notably, m_h is generated by the LLM based on its understanding of the quest descriptions and auxiliary information and primarily serves as guiding reference information for the subsequent planning phase, input back into the LLM. LLM utilizing a large language model to derive M from the inputs Q, I, and P. P is the prompt serving as a structured input designed to guide the LLM in generating objectives. It provides contextual information and instructions that help the LLM align its output with the requirements of the game environment and the specific objectives to be achieved.

This abstraction simplifies the solution space by reducing the problem's dimensionality while retaining the flexibility required for high-freedom quests. The methods act as guiding nodes in a broader quest graph, structuring exploration and ensuring systematic coverage of possible paths.

C. Step2: Primitive Task Generation and Input Mapping

Based on methods, the framework uses LLMs to generate general primitive tasks that serve as the fundamental units for planning and are mapped into actionable game input actions.

Directly using LLMs to generate every specific game action is inefficient. In grid-based RPG games, for instance, many player inputs involve repetitive actions, such as moving between locations, which do not require advanced reasoning. Generating such actions with LLMs increases computational costs without significant benefits. Furthermore, as LLMs rely on text-based inputs and outputs, they are better suited for generating discrete inputs, limiting their applicability to games requiring continuous inputs.

To address these challenges, the framework generates primitive tasks instead of direct game input actions. By combining with game development information, primitive tasks are efficiently mapped to game input actions. This method reduces computational overhead and expands the framework's applicability to both discrete and continuous input games.

1) Primitive Task Generation: Primitive tasks are designed for composability, enabling methods to be achieved by combining and arranging these general-purpose building blocks. Each primitive task is defined as a structured element to ensure clarity, reusability, and compatibility with subsequent primitive task sequence planning and input mapping processes. Formally, the primitive task generation process can be expressed as:

$$(Q, I, M, P) \xrightarrow{\text{LLM}} PT = \{pt_l \mid l = 1, \ldots, m\}$$

where $PT = \{pt_l\}$ is the set of primitive tasks. Each $pt_l \in PT$ is a general-purpose task designed to be composable and flexible. These primitive tasks act as reusable building blocks that can be combined to achieve various methods. I is auxiliary information that includes game design introductions and other contextual information related to the instruction.

Each primitive task $pt_l \in PT$ is defined as a triplet:

$$pt_l = \langle \text{Desc}_l, \text{Pre}_l, \text{Effect}_l \rangle$$

where Desc_l is a textual description of the task, typically consisting of a verb and a target noun (e.g., "put the dish on the table"), ensuring the basic and general nature of the tasks. Pre_l represents the preconditions that must be satisfied before execution to ensure the validity of the task (e.g., "player must have a dish"), and Effect_l denotes the effects after completing the primitive task, providing a reference for subsequent steps (e.g., "dish on the table").

This structured representation ensures that each pt_l is clear, reusable, and compatible with subsequent steps. Moreover, while Pre_l and Effect_l provide important guidance for planning, they act as soft constraints rather than strict requirements, ensuring flexibility in action planning and execution.

2) Input Mapping: Once primitive tasks are generated, they are mapped to actionable game input actions through a mapping function f_{pt_l}. This mapping function is generated by the LLM based on the primitive task pt_l and auxiliary game development information I. Formally, the mapping function can be expressed as:

$$(pt_l, I, P) \xrightarrow{\text{LLM}} f_{pt_l}$$

For each $pt_l \in PT$, the mapping function f_{pt_l} translates the current game state s into a sequence of game input actions:

$$f_{pt_l} : s \rightarrow \mathcal{A}_l = [a_1, a_2, \ldots, a_k]$$

where s is the current game state. $\mathcal{A}_l$ is the sequence of game input actions corresponding to pt_l.

For example, the primitive task "move to target location" might utilize pathfinding algorithms, such as A* or Dijkstra, to compute the optimal route from the player's current position to the target location. This route is then decomposed into specific game input actions (e.g., "move forward," "turn right") and combined with additional commands (e.g., "pick up," "use") to produce complete action scripts.

This approach enables the framework to handle a variety of game input action types while ensuring logical consistency and execution efficiency.

D. Step3: Primitive Task Sequence Generation

After generating the primitive tasks, the system utilizes LLMs to generate detailed task sequences for each method. These sequences serve as actionable plans to accomplish the specified methods.

The sequence generation process can be formalized as:

$$(s_t, Q, m_h, PT, P) \xrightarrow{\text{LLM}} \text{Sec}_{m_h}$$

where s_t is the current game state, Sec_{m_h} is the primitive task sequence generated for the method m_h. The sequence consists of a set of primitive tasks:

$$\text{Sec}_{m_h} = \{pt_{l,1}, pt_{l,2}, \ldots, pt_{l,k}\}$$

Here, $pt_{l,t'}$ represents the t'-th primitive task chosen to accomplish the method m_h.

To ensure the accuracy of the generated action sequences, we incorporate Chain-of-Thought (CoT) [53] reasoning and a Reflection Mechanism [54] as prompt engineering techniques to optimize the LLM's output.

CoT guides the model to reason step-by-step, helping it establish logical relationships between actions and avoid reasoning jumps. Specifically, when generating each $pt_{l,t'}$, the LLM is prompted to simultaneously produce three elements: first, a primitive task $pt_{l,t'}$, which is a specific objective description indicating the task to be achieved (e.g., "move to the kitchen"); second, a state analysis $s'_{t'}$, which details how this action affects the current game state and any resulting changes (e.g., "player reaches the kitchen"); and finally, a next-step plan $P_{t'+1}$, which outlines the subsequent action or strategy based on the updated state and the ongoing objective (e.g., "open the refrigerator and take out the lettuce").

Through CoT, the LLM progressively derives various stages of the quest, ensuring that each task aligns with the objective. This structure allows the LLM to consider the current state while anticipating the consequences of each action, improving the accuracy of sequential action planning.

Additionally, to further enhance the accuracy and effectiveness of the primitive task sequences, we introduce a Reflection Mechanism to refine the task plan. This mechanism follows three sequential steps. First, the LLM generates an initial sequence of primitive tasks based on the given task description and the current state. Second, it undergoes a reflection phase, during which the initial sequence is carefully analyzed to identify and pinpoint any logical inconsistencies, inefficiencies, or errors. Finally, in the revision phase, the LLM revises and optimizes the initial sequence based on the issues identified during reflection, ensuring that the resulting task plan is more accurate, coherent, and efficient.

By combining CoT reasoning with the Reflection Mechanism, the system ensures the accuracy and coherence of action planning. This approach not only improves the quality of the initial objective sequences but also refines them iteratively, enhancing task completion efficiency and success rates. Ultimately, this mechanism ensures that the execution order and state transitions align with expectations, maximizing the accuracy of quest execution in complex scenarios.

For each method, multiple primitive task sequences are generated to enrich the state-action transition graph construction,

and by leveraging the mapping function f_{pt_l} associated with each $pt_{l,t'}$ in the sequence Sec_{m_h}, these sequences are translated into corresponding game action inputs for interaction with the game environment.

E. Step4: State-Action Transition Graph Construction

As the system executes action sequences, it dynamically constructs a state-action transition graph that captures all explored game states and their transitions. In this graph, each node represents a unique game state s_i, and is encoded as a fixed-length vector $\mathbf{s_i} \in \mathbb{R}^d$ to ensure distinct representation. The state vector $\mathbf{s_i}$ encapsulates all the relevant information about the state at any given point in the game, ensuring that each state is uniquely identifiable.

Edges in the graph represent the actions a_i that transition between game states. Formally, a state transition can be described as:

$$s_i \xrightarrow{a_i} s_{i+1}$$

The state-action transition graph serves several potential purposes in the framework:

- Path Analysis: The graph allows the system to identify all feasible paths from the initial state s_0 to the quest completion state $s_n \in E$. Specifically, the system can trace different sequences of actions $\{a_1, a_2, \ldots, a_n\}$ that lead to a final objective state $s_n \in E$. The total number of such paths can be computed by traversing the graph and counting all unique sequences of edges from s_0 to $s_n \in E$.
- Redundancy Elimination: As the system explores the game space, redundant or invalid paths that do not lead to quest completion or that revisit previously explored states can be filtered out. This ensures more efficient exploration by removing circular paths or states that do not contribute to objective achievement.
- Multi-Ending Coverage: For quests with multiple possible outcomes, such as reaching different destinations in a game, the state-action transition graph highlights distinct paths leading to each possible endpoint. For instance, a player might be tasked with traveling to either a forest or a mountain, with each destination requiring different actions, such as selecting specific routes or gathering necessary resources along the way. The graph captures these variations by mapping out the unique steps required for each destination, dynamically adapting to the quest's requirements. By reflecting different choices and their resulting paths, the graph ensures a comprehensive exploration of all possible outcomes, enabling thorough testing and validation of multi-ending scenarios.
- Enhanced Interpretability: Beyond its abstract vector representations, the state-action transition graph can be integrated with visualization tools to support reverse mapping from state vectors back to their corresponding in-game snapshots. By complementing each node with visual cues (e.g., game screen captures) or descriptive state information, developers can more intuitively identify and interpret each state and its transitions. This visual linkage helps developers better understand gameplay, identify issues, and communicate results, making debugging more efficient.

F. Prompt Design

Figure 2 shows examples of prompts used in this study. We systematically design the LLM prompts by considering the following key elements.

Game Information and Quest Background. The primary task of the prompts is to ensure that the LLM receives sufficient game information and quest context to correctly interpret the quest and make appropriate decisions. Game information includes the game type, quest objectives, available resources, and player states, all of which help the LLM quickly understand the game environment before starting the quest. When designing the prompts, we distinguish between two perspectives: the developer's perspective and the player's perspective. The developer's perspective focuses on the technical details of the game, such as quest structure, level design, and possible action sequences, while the player's perspective focuses on the player's actual interactions with the game environment, assisting the LLM in generating action objectives and input mappings that align with the player's needs. This distinction in perspectives ensures that the LLM can generate contextually accurate outputs at different stages of the quest. In the action objective generation phase, the developer's perspective helps the LLM generate valid and reliable objectives to support subsequent testing. In the action sequence generation phase, the player's perspective provides necessary contextual information, helping the LLM accurately plan the sequence of actions and simulate player behavior.

Prompt Instructions. The instructions within the prompt are key to guiding the LLM in generating the action sequences. The instructions vary according to the objectives of different stages in the quest. To ensure the LLM accurately understands the quest and generates high-quality steps, the prompts must possess several key characteristics: first, they should be clear and concise, avoiding unnecessary complexity to reduce comprehension errors; second, tasks should be broken down into stages to allow the LLM to work step-by-step and avoid information overload; third, the prompts must be closely aligned with the game environment, ensuring that the actions generated by the LLM are consistent with the actual game state, resources, and objectives. Furthermore, the introduction of CoT and Reflection mechanisms can significantly improve the quality of quest generation. CoT helps the LLM verify the necessity of each action through step-by-step reasoning, reducing potential errors in quest planning. The reflection mechanism allows the LLM to identify and correct logical flaws during the generation process, improving the overall accuracy and coherence of the generated sequence.

Output Format Control. To ensure that the LLM's output is structured, analyzable, and efficient for subsequent processing, we impose strict output format requirements in the prompts. Each stage of the quest output should follow a consistent format, particularly in the generation of primitive tasks and action sequences, where the output is presented

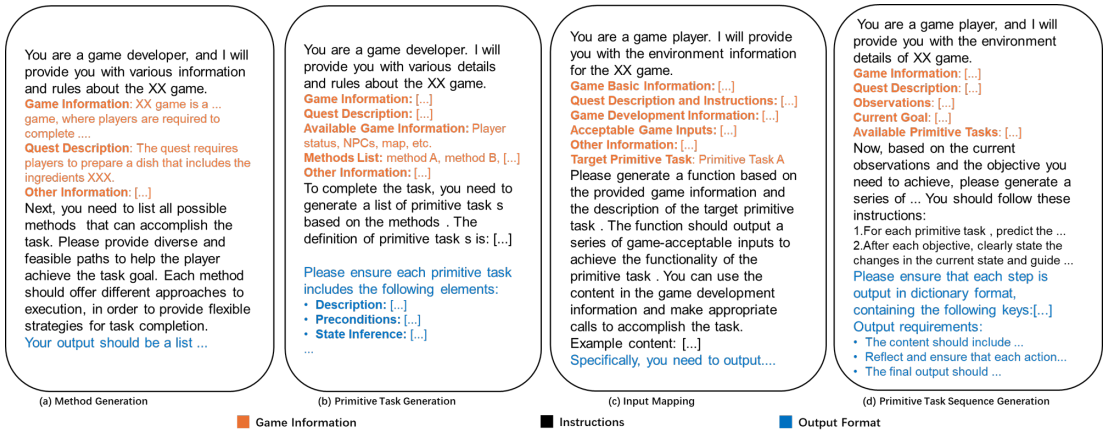


Fig. 2: Prompt example for the EndWeaver

in a dictionary structure. The dictionary must contain key information such as "Action Description," "State Analysis," and "Next Step Planning," clearly indicating the purpose, preconditions, and expected outcomes of each operation for subsequent analysis and execution. This format control not only ensures the logical coherence and traceability of quest execution but also enables effective quest analysis and testing through structured output data.

V. Evaluation

This evaluation is designed to address the following research questions (RQ):

- RQ1: How consistently can EndWeaver complete quests, and how concise are its successful action plans, compared to baseline methods?
- RQ2: How effectively can EndWeaver discover and cover all designated quest endings?
- RQ3: How does EndWeaver compare with baseline methods in terms of the total number of environment interaction steps required during the testing process?

A. Experimental Setting

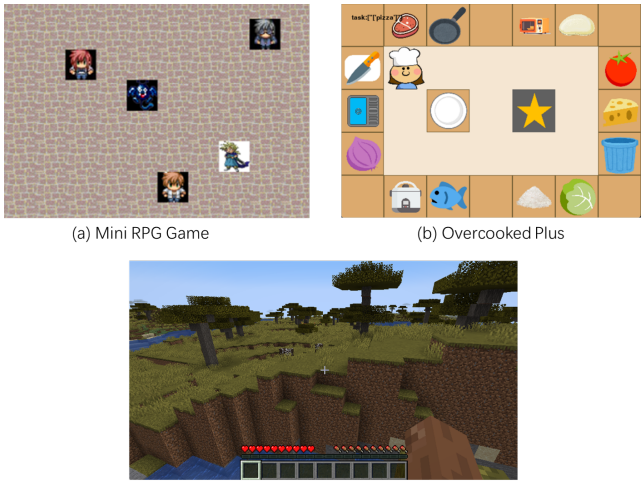
Evaluation Metrics. We adopt the following quantitative metrics for the above research questions. For RQ1, we adopt two key metrics. The first is the completion rate, which measures the proportion of test runs in which the quest is successfully completed. The second is the average number of steps required to complete the quest, calculated only over successful runs, which reflects the length of the action path needed to achieve completion. For RQ2, we count the number of times each possible ending (Ending A, B, or C, with increasing difficulty) is achieved among all successful runs, and compute their corresponding proportions. For RQ3, we evaluate the total number of environment interaction steps incurred throughout the entire testing process, including both training and testing phases.

Baseline Methods. We select four representative baselines. All methods share the same action and observation spaces to ensure fair comparison: (i) Random Algorithm [39], [55]: Randomly samples actions from the available action space until the quest is completed or the maximum step limit is reached, serving as a difficulty benchmark for uninformed exploration. The maximum step limit is 5,000 steps for Mini RPG, 10,000 steps for Overcooked Plus and 1,000 steps for Minecraft. (ii) Genetic Algorithm (GA) [56]: A classic search-based method that evolves fixed-length action sequences to optimize quest completion. The sequence length is 100 for Mini RPG, 1,000 for Overcooked Plus, and 50 for Minecraft; the population size is 50; the maximum number of generations is 150; the crossover rate is 0.7, and the mutation rate is 0.5. An early stop mechanism is applied during training to terminate evolution when no improvement is observed over a fixed number of generations. Each test run includes a complete GA training–execution process, and the best individual is selected for evaluation. (iii) Proximal Policy Optimization (PPO) [57]: A widely used policy-gradient reinforcement learning algorithm that optimizes a policy network to maximize expected return. We use vectorized game states as input and design environment-specific reward functions. Training steps are fixed at 500,000 for each experiment with an early stop mechanism. The reward settings are as follows: Mini RPG: step penalty -0.1, subquest completion reward $+100$; Overcooked Plus: step penalty -0.1, ingredient processing reward $+10$, quest completion reward $+1000$, incorrect submission penalty -20; Minecraft: step penalty -0.1, obtaining required item reward $+10$, quest completion reward $+1000$. Each test run consists of a full training–execution process, and inference uses stochastic action sampling to encourage exploration. (iv) ELLM-PPO [52]: An enhanced PPO method that introduces LLM-assisted exploration. GPT-4o generates natural language subgoals based on the current state description and quest specification. An intrinsic reward is computed as the semantic similarity between state transition descriptions and generated subgoals. The agent is pretrained with this augmented reward and then fine-tuned with standard PPO. Training settings are kept identical to the PPO baseline for fairness. (v)EndWeaver (Ours): Uses GPT-4o as the LLM planner, combined with a multi-level planning structure and a state–action transition graph to solve quests. For each method, 30 action sequences are generated and recorded in the state-action transition graph. The graph is then processed

by merging identical nodes and removing redundant paths, after which 10 valid quest completion trajectories are randomly sampled as the final output.

In the experiment, we adopt a unified evaluation protocol across all environments and methods to ensure the robustness of statistical results. In each experiment, all quests are tested with all methods for 10 independent trials. The entire process is repeated 30 times, and all reported values are presented as mean $\pm$ 95% confidence interval, constructed using the t-distribution for means and the Wilson score interval for proportions.

B. Game Environments



(a) Mini RPG Game (b) Overcooked Plus

(c) Minecraft

Fig. 3: Game Image.

To comprehensively evaluate our method, we conduct experiments across three distinct game environments: Mini RPG, Overcooked Plus, and Minecraft. These environments are chosen to cover a diverse range of game genres, quest structures, and difficulty characteristics.

Mini RPG Game. Mini RPG is a simplified, linear, story-driven role-playing game [39], designed to evaluate quest understanding and planning in natural language–rich scenarios. It preserves the core logic of traditional RPGs while simplifying the map and interactions to support rapid quest generation and reproducibility. The environment consists of a fixed grid map containing NPCs, monsters, the player character, and interactive terrain. Players can interact with NPCs, collect or submit items, engage in combat, and trigger scripted events. The game state is fully observable and represented as a structured dictionary that includes the map layout, character attributes, NPCs, and active quests. The environment features 9 quests, each with multiple completion routes of varying difficulty. Each route consists of a sequence of subquests described in natural language, requiring the player to follow different action sequences. Mini RPG Game, as a simple and basic environment, is designed to provide an entry-level testing scenario for the method, verifying whether it can stably achieve objectives in low-complexity quests and establishing a performance baseline for subsequent evaluation in more complex environments.

Overcooked Plus. Overcooked Plus [58] is an open-source and difficulty-enhanced version of the commercial cooking game Overcooked [59], designed to evaluate planning under operational constraints and execution sensitivity. Players prepare dishes by processing and combining ingredients through actions like chopping, cooking, plating, and serving. The environment enforces strict rules—for example, each station holds only one item, and incorrect placements must be explicitly discarded. It is fully observable, with structured state representations that track the location and status of all relevant objects. The quest set includes 7 cooking quests, each allowing multiple valid completion strategies reflecting increasing difficulty levels. Harder completions typically require more complex combinations and precise action sequencing. This environment emphasizes sequential planning under strict rules, requiring the agent to maintain high execution accuracy while rapidly responding to state changes and correcting errors.

Minecraft. Minecraft [60] is a large-scale sandbox environment widely used in AI research due to its open-ended structure and rich combinatorial interaction dynamics. To support consistent evaluation, we adopt a high-level action abstraction within the EndWeaver framework, where all methods issue discrete, parameterized commands such as "mine," "craft," or "move." These are internally mapped to low-level atomic operations. The environment provides full observability of the agent's position, inventory state, resource locations, and tool status. To ensure comparability across methods, we discretize the action space into a fixed set of parameterized high-level commands. We define 11 quests inspired by Minecraft's achievement system, spanning resource gathering, item crafting, and combat. Each quest supports multiple valid solutions of varying complexity, encouraging long-horizon planning and adaptive strategy. This environment is the most challenging among the three and is closest to real-world settings, with its difficulties stemming primarily from long dependency chains and a vast state space.

C. Experiment Results

Table I summarizes the performance of all evaluated methods across the three game environments. In terms of quest completion rate, EndWeaver achieves well results across all environments, significantly outperforming all baseline methods. In contrast, Random performs the worst due to its uninformed random action strategy and inability to exploit the environment structure. GA and PPO, as two traditional methods, exhibit environment-dependent performance with limited stability. ELLM-PPO, which incorporates LLM-assisted intrinsic rewards, consistently outperforms GA and PPO across multiple settings, demonstrating improved generalization. However, its overall performance remains noticeably lower than that of EndWeaver.

Regarding average completion steps, EndWeaver also demonstrates a clear advantage. In Mini RPG, it achieves quest completion with an average of only 28.3 steps, significantly lower than all baselines. In Overcooked Plus and Minecraft,

TABLE I: Performance Metrics for Quest Completion and Ending Diversity across Three Game Environments

Game	Method	Comp. Rate(%)	Avg Steps	Ending A(%)	Ending B(%)	Ending C(%)	Total Steps
Mini RPG	Random	25.3 ± 3.8	1463.9 ± 155.2	66.7 ± 5.1	24.4 ± 4.5	8.9 ± 2.3	1,001,046
	GA	54.4 ± 4.5	81.0 ± 7.8	58.5 ± 5.2	24.5 ± 4.1	17.0 ± 3.3	124,086,117
	PPO	62.5 ± 4.8	47.2 ± 3.5	74.2 ± 4.1	18.4 ± 3.2	7.4 ± 2.6	10,838,016
	ELLM-PPO	74.6 ± 3.9	53.2 ± 4.9	65.0 ± 4.4	22.1 ± 3.6	13.0 ± 2.9	9,920,000
	ENDWEAVER (ours)	100.0 ± 0.0	28.3 ± 1.6	40.6 ± 3.3	30.7 ± 3.1	28.7 ± 2.8	6,750
Overcooked Plus	Random	49.2 ± 5.1	5686.2 ± 481.6	70.6 ± 5.8	29.4 ± 5.8	0.0 ± 0.0	688,432
	GA	68.5 ± 4.6	753.1 ± 68.3	47.1 ± 5.5	42.9 ± 5.6	10.0 ± 3.1	17,854,231
	PPO	56.2 ± 5.3	43.8 ± 4.1	88.8 ± 2.8	9.3 ± 2.3	1.9 ± 0.9	2,905,484
	ELLM-PPO	78.4 ± 4.1	79.3 ± 7.2	56.5 ± 5.1	33.6 ± 4.4	9.9 ± 2.8	3,510,323
	ENDWEAVER (ours)	100.0 ± 0.0	66.0 ± 3.5	23.2 ± 3.6	46.4 ± 4.3	30.4 ± 3.0	15,036
Minecraft	Random	32.1 ± 4.4	462.4 ± 91.5	74.7 ± 6.3	25.3 ± 6.3	0.0 ± 0.0	431,456
	GA	55.6 ± 5.2	15.3 ± 2.4	62.5 ± 6.1	25.0 ± 4.8	12.5 ± 3.3	5,262,322
	PPO	43.7 ± 5.5	6.9 ± 0.9	89.9 ± 3.1	7.4 ± 2.4	2.7 ± 1.2	335,163
	ELLM-PPO	68.8 ± 4.4	8.6 ± 1.2	69.4 ± 5.5	19.0 ± 4.1	11.6 ± 3.2	293,112
	ENDWEAVER (ours)	100.0 ± 0.0	7.3 ± 0.6	37.0 ± 4.5	34.3 ± 4.2	28.7 ± 3.8	1,413

Note: Ending difficulty is defined by the minimal number of actions required to reach it. Longer sequences indicate higher difficulty.

although PPO yields lower average step counts (43.8 and 6.9, respectively), a closer examination of the ending distribution reveals a strong bias toward the simplest outcome, Ending A, in both PPO and ELLM-PPO. For example, in Minecraft, PPO achieves Ending A in 89.9% of successful runs, while the hardest outcome, Ending C, is achieved in only 2.7%. Since ending difficulty in our evaluation is defined by the required action sequence length, a lower average step count does not necessarily indicate more efficient or effective quest planning. By contrast, ENDWEAVER achieves a more balanced distribution across all three endings while maintaining relatively low average steps, indicating that it can complete quests with concise action paths.

In terms of ending coverage, the differences between methods are further amplified in the more challenging environments. While most methods can effectively reach all three endings in the relatively simple Mini RPG game, performance diverges significantly in Overcooked Plus and Minecraft. Random fails to reach any Ending C, and PPO-based methods display a strong bias toward simple paths. GA shows moderate ability to reach high-difficulty outcomes (e.g., 12.5% for Ending C in Minecraft). In contrast, ENDWEAVER achieves 30.4% coverage of Ending C in Overcooked Plus and attains a near-uniform distribution across all endings in Minecraft (37.0%, 34.3%, and 28.7% for Endings A, B, and C, respectively). These results suggest that ENDWEAVER possesses robust multi-ending testing capability and can reliably discover diverse completion strategies across varying difficulty levels.

With respect to testing cost, ENDWEAVER incurs dramatically fewer total environment interaction steps than all baselines. It requires only 6,750 steps in Mini RPG, 15,036 steps in Overcooked Plus, and 1,413 steps in Minecraft. By comparison, GA demands over 124 million steps in Mini RPG, and PPO consumes over 10 million steps. Even ELLM-PPO, which benefits from LLM-guided exploration, requires nearly 10 million steps. This several-orders-of-magnitude gap highlights the exceptional interaction efficiency of ENDWEAVER, which maintains high performance with minimal interaction overhead.

D. Ablation Study

To systematically evaluate the contribution of each core component in ENDWEAVER, we conduct an ablation study in Overcooked Plus. Specifically, we investigate the roles of three key modules: (i) LLM;(ii) State-Action Transition Graph (Graph); and (iii) Multi-level Planning (MLP).

Ablation on LLMs. To assess the impact of different LLMs, we keep the full framework intact ("Full" setting, retaining both Graph and MLP) and use different LLMs including GPT-4o, GPT-4o-mini, Qwen2-72B, DeepSeek-V2, and Claude-3.5-Haiku for comparison (other experimental settings remain identical to those in Section V-A). The results reveal a clear performance stratification: the first-tier models (e.g., GPT-4o, DeepSeek-V2, Qwen) not only complete all quests stably but also discover all endings, including the most complex Ending C; second-tier models (e.g., GPT-4o-mini, Claude-3.5-Haiku) can still achieve a 100% completion rate under the framework's guidance but fail to reach Ending C entirely. This indicates that while the robustness of ENDWEAVER ensures a strong lower bound — enabling even weaker LLMs to complete basic quests — the ability to tackle the most challenging reasoning problems and explore the deepest endings is fully determined by the reasoning capability of the underlying LLM.

Ablation on Graph (w/o Graph). In the "w/o Graph" setting, the system directly samples action sequences from the LLM's outputs without constructing a state-action transition graph. As shown in Table II, removing the Graph leads to a significant performance drop: even the strongest model, GPT-4o, sees its completion rate fall from 100.0% to 46.1%, highlighting the critical role of the Graph in ensuring stable and consistent quest execution. The coverage of complex endings also deteriorates markedly without the Graph, with completion rates for the most difficult Ending C dropping sharply — in some models, to zero. This occurs because the state-action transition graph enables the merging and reuse of shared subpaths, producing a richer and more diverse set of feasible trajectories,

TABLE II: Performance comparison of different settings in Overcooked Plus

Setting	Method	Comp. Rate(%)	Avg Steps	Ending A(%)	Ending B(%)	Ending C(%)	Total Steps
Full (Ours)	DeepSeek-V2	100.0 ± 0.0	69.2 ± 4.0	20.8 ± 3.9	52.0 ± 5.2	27.2 ± 4.0	16,184
	Qwen	100.0 ± 0.0	77.7 ± 4.2	25.3 ± 4.1	41.8 ± 5.1	32.9 ± 4.5	15,428
	GPT-4o	100.0 ± 0.0	66.0 ± 4.2	23.2 ± 4.4	46.4 ± 5.5	30.4 ± 4.7	15,036
	GPT-4o-mini	100.0 ± 0.0	36.4 ± 2.1	42.0 ± 10.0	58.0 ± 10.0	0.0 ± 0.0	14,951
	Claude-3.5-Haiku	100.0 ± 0.0	33.5 ± 1.4	57.0 ± 10.0	43.0 ± 10.0	0.0 ± 0.0	15,363
w/o Graph	DeepSeek-V2	26.1 ± 4.3	79.8 ± 4.3	64.3 ± 9.7	20.7 ± 8.6	15.6 ± 8.5	16,149
	Qwen	28.1 ± 5.6	81.2 ± 2.9	45.8 ± 10.8	50.6 ± 9.1	3.6 ± 3.5	15,182
	GPT-4o	46.1 ± 5.2	82.9 ± 2.6	25.7 ± 9.5	59.6 ± 9.4	16.3 ± 7.6	14,823
	GPT-4o-mini	5.6 ± 2.3	69.8 ± 4.1	90.7 ± 12.2	10.6 ± 13.8	0.0 ± 0.0	14,781
	Claude-3.5-Haiku	27.5 ± 5.7	69.0 ± 2.3	32.4 ± 9.6	67.2 ± 10.4	0.0 ± 0.0	15,216
w/o MLP	DeepSeek-V2	100.0 ± 0.0	26.0 ± 1.0	100.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	6,128
	Qwen	100.0 ± 0.0	26.6 ± 1.2	100.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	5,953
	GPT-4o	100.0 ± 0.0	25.8 ± 0.6	100.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	6,761
	GPT-4o-mini	100.0 ± 0.0	25.3 ± 0.4	100.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	5,888
	Claude-3.5-Haiku	100.0 ± 0.0	26.4 ± 1.4	100.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	6,544

especially for deeper or less probable endings. Additionally, the average steps required to complete quests increase significantly without the Graph. Lacking global knowledge of explored states and transitions, the system tends to generate longer or redundant sequences. In contrast, the Graph allows for path pruning and reuse, facilitating the generation of shorter and more efficient plans. These findings demonstrate that the transition graph is essential not only for increasing execution stability, but also for enabling efficient trajectory construction and deeper exploration within the solution space.

Ablation on Multi-level Planning (w/o MLP). In the "w/o MLP" setting, the LLM generates entire action sequences solely based on the quest description and game instructions, which are then recorded into the state-action transition graph, with no hierarchical goal decomposition and prompt-based completion preference guidance. The results show that all five LLMs maintain a 100.0% completion rate without MLP; however, ending diversity collapses entirely, with all runs converging to the simplest Ending A (B and C coverage: 0.0%). The average steps also drop significantly to around 25–26. For instance, GPT-4o achieves Avg Steps / Total Steps of 25.8 / $6{,}761$ under "w/o MLP", compared to 66.0 / $15{,}036$ in the full framework, with the ending diversity shifting from $A/B/C = 23.2/46.4/30.4$ to $A = 100.0$, $B = C = 0$. DeepSeek-V2 and Qwen show the same trend. It is important to emphasize that this reduction in average steps does not indicate improved efficiency; instead, it reflects a shift in difficulty distribution and a metric bias caused by exclusively exploring the simplest ending. This phenomenon is analogous to the PPO results discussed in Section V-C. Therefore, the multi-level planning structure, through hierarchical goal decomposition and path guidance, effectively drives the system into longer and more complex dependency chains, thereby unlocking medium/high-difficulty branches (B/C) and improving multi-ending coverage.

E. Discussions

Across all three testing environments, EndWeaver demonstrates consistently superior performance, maintaining a perfect quest completion rate of 100.0% while covering all designated quest endings. Moreover, it achieves this with orders-of-magnitude fewer environment interaction steps compared to baseline methods. Even in the highly complex and long-horizon Minecraft environment, this advantage remains evident. As shown in Table I, EndWeaver not only completes quests reliably but also generates concise yet diverse action paths, achieving a balanced distribution across easy, medium, and hard endings—a capability rarely seen among baseline approaches.

The ablation studies further highlight the contribution of each core component. The reasoning capability of the LLM sets the upper bound of the system's performance: stronger models consistently discover more complex endings, whereas smaller-scale models, though capable of maintaining high completion rates under the framework's guidance, fail to reach high-difficulty branches. The state–action transition graph is essential for both execution stability and deep exploration; removing it leads to sharp declines in both completion rate and outcome diversity, as the agent loses the ability to reuse subpaths, prune redundant exploration, and globally coordinate trajectories. The multi-level planning structure is also critical for multi-ending coverage, as all runs collapse into the simplest ending with no MLP.

In contrast, traditional RL methods (e.g., PPO) and search-based approaches (e.g., GA) remain fundamentally constrained by their "search-style" strategy paradigm, with their performance heavily reliant on the design of reward functions. These approaches suffer from the sparsity and externality of reward signals—PPO in particular tends to fall into local optima. In the Overcooked Plus, PPO often converges to the simplest ending. We hypothesize that one key bottleneck lies in the limitations of its reward function design. This issue is especially pronounced in long-horizon, high-complexity quests, where intermediate actions crucial for reaching complex endings (e.g., backtracking) may not yield immediate rewards, making it difficult for policy networks to learn them. As a result, PPO struggles to explore the full state space leading to harder endings. Crafting such fine-grained reward structures requires

deep domain knowledge and extensive trial-and-error, making them difficult and costly to implement in complex game scenarios. It's worth noting that the costs of tuning reward functions are not included in the results shown in Table I, and including them would further increase total overhead. ELLM-PPO, which leverages LLM-assisted intrinsic rewards, alleviates local optimum issues to some extent but is still fundamentally bound by the limitations of RL's exploration paradigm. GA achieves slightly better diversity than PPO due to its global search nature, but at the cost of prohibitively high sample complexity and computational expense.

To address these limitations, ENDWEAVER takes a fundamentally different perspective, grounded in semantic-level planning rather than reward-driven search. By explicitly reasoning about the final objectives and their logical dependencies before execution, ENDWEAVER can generate purposeful and diverse action trajectories from a global perspective, effectively avoiding the local optima that plague RL-based methods. Furthermore, this architecture offers strong scalability: when adapting to new environments or quests, only the game mechanics, APIs, and mappings between primitive tasks and game operations need to be specified—forming a knowledge-engineered pipeline—rather than retraining costly reward functions or policy networks. This will potentially lower migration and deployment costs while maintaining stable, high performance in long-horizon, complex quests.

Beyond serving as an effective tool for initial smoke testing, our framework also has the potential to provide lasting value throughout the entire game testing lifecycle. The constructed state–action transition graph encapsulates rich knowledge of quest structures and player–environment interactions, which can be reused and extended for diverse testing activities. For instance, in regression testing, previously discovered key paths and endings can be quickly replayed to validate whether critical functionalities remain intact after game updates, reducing costs by several orders of magnitude compared to full re-exploration. In test coverage analysis, the graph enables systematic inspection of explored and unexplored branches, guiding the prioritization of future testing. Moreover, in quest design verification, the graph—combined with visualization—offers designers a transparent overview of the strategy space, helping to identify unintended shortcuts, dead ends, or logical flaws. Taken together, these capabilities suggest that the knowledge constructed by our framework has the potential to evolve into a lifecycle-through testing assets, extending its value beyond smoke testing.

F. Limitations

While the experimental results demonstrate the effectiveness and robustness of ENDWEAVER, several limitations remain that warrant discussion and provide directions for future improvement.

First, the multi-level planning in ENDWEAVER is highly dependent on the availability of quest descriptions as input. If the descriptions are ambiguous, incomplete, or do not clearly reflect the existence of multiple endings, the system's ability to explore diverse paths can be significantly constrained. Therefore, providing high-quality and well-structured quest descriptions is a fundamental prerequisite for unlocking the full potential of ENDWEAVER.

Second, the quests used in this study are relatively ordinary and rely primarily on basic logic and common-sense knowledge—both of which are well-covered during the pretraining of general-purpose LLMs. This contributes to the system's stable performance in such settings. However, when applied to more complex or domain-specific scenarios, the reasoning capability of the underlying LLM may become a bottleneck, negatively impacting overall performance. Beyond capability limitations, cost is another important consideration. Although our experiments primarily report environment interaction costs, the actual runtime cost of ENDWEAVER also includes the computational overhead of LLM calls. This cost can become non-trivial in large-scale applications. Since both performance and cost are closely tied to the quality of the base model and its alignment with the quest, future deployments must strike a balance between the two. For instance, smaller, locally deployed models fine-tuned with domain-specific data could be used to reduce inference costs while maintaining sufficient planning capability.

Third, as the complexity of games and quests increases, the state–action transition graph must be abstracted to avoid handling redundant or overly fine-grained state details, while still preserving critical nodes and connectivity. However, when the number of interactive objects and quest states grows dramatically, the graph structure can rapidly expand in size if no effective state equivalence abstraction mechanism is employed. This may lead to significant degradation in storage, retrieval, and sampling efficiency, posing further scalability challenges for the system.

G. Threats to Validity

For internal validity, the first concern lies in the inherent randomness of LLM outputs, which may affect reproducibility and performance stability. To address this, we perform each test multiple times under identical settings and report mean values with 95% confidence intervals. Second, the pretraining data of LLMs is not transparent and may contain knowledge overlapping with certain quests or environments, thereby introducing a potential evaluation bias. To mitigate this, we use multiple LLMs from different vendors (e.g., GPT-4o, Qwen, DeepSeek) to reduce dependence on any single source of prior knowledge. Third, the performance of baseline methods may be affected by the design of reward functions and hyperparameters, while the suboptimal configurations could lead to unfair comparisons. To minimize this risk, we designed competitive reward functions and parameter settings in line with best practices.

For external validity, one key concern is the restricted scope of evaluated environments. We test on three games—Mini RPG, Overcooked Plus, and Minecraft—selected to represent diverse genres and quest structures. However, these still cannot fully reflect the wide variety of real-world games, such as FPS, puzzle, or narrative-heavy adventure games. Additional evaluations are needed to validate generalizability

in future work. Another threat concerns the realism of quest descriptions. Although we designed the quests by drawing inspiration from actual commercial games and incorporating multi-ending mechanics, the descriptions themselves are still manually constructed. Real-world game quests—especially in industrial settings—may involve noisier, less structured, or intentionally obfuscated information.

VI. Conclusion and Future Work

This paper introduces a novel framework based on LLMs, EndWeaver, for automated smoke testing of multi-ending game quests. By integrating multi-level planning with a dynamic state-action transition graph, the framework effectively addresses the challenges of path exploration and efficiency in multi-ending game smoke testing. In multi-level planning, high-level method ensures broad exploration of quest paths, while low-level primitive task provides more granular planning units, enabling more precise task execution and comprehensive coverage of diverse gameplay scenarios. The dynamic state-transition graph records the relationships between states and actions during quest execution, supporting comprehensive path coverage while eliminating path redundancy, and providing a foundation for subsequent testing.

In the experimental evaluation, we assess the effectiveness of the EndWeaver framework across three distinct game environments: Mini RPG, Overcooked Plus, and Minecraft. We compare EndWeaver with five baseline methods — Random, Genetic Algorithm, Proximal Policy Optimization, and ELLM-PPO. The results show that EndWeaver outperforms these methods in task completion rates, ending diversity, and gameplay interaction efficiency. Additionally, we conducted ablation studies to assess the contribution of multi-level planning to comprehensive exploration and the role of the graph structure in path generation. Furthermore, by comparing different LLMs, we confirmed a positive correlation between the framework's performance and the capabilities of the LLMs used.

Although the framework has shown promising results, further exploration and improvement are required in the following directions. First, the primary focus of future work will be to closely collaborate with industry partners to evaluate the framework's effectiveness in real-world industrial settings. The environments used in the current experiments are still relatively simple, and evaluating the framework in actual industrial scenarios will help assess its practical applicability and ensure its relevance in more complex, real-world game development contexts. Second, support for dynamic environments is another area for future development. The current EndWeaver framework is primarily designed for relatively static environments. However, in more complex industrial games, particularly online games, game objects and states may change unpredictably. Therefore, future work will focus on integrating real-time monitoring and adaptive planning strategies. This will ensure that the framework can maintain stable and effective testing even in highly dynamic and unpredictable environments. Finally, another potential area for future research is supporting regression testing. Regression testing is crucial for ensuring the quality of games during long-term operations, as it ensures that new features or fixes do not break existing game content or quest designs. The graph structure built in EndWeaver is particularly well-suited for use in regression testing. Thus, future work could extend the current framework to support regression testing, enhancing its adaptability and ensuring continuous and effective quality assurance throughout the entire development lifecycle.

References

[1] S. Värtinen, P. Hämäläinen, and C. Guckelsberger, "Generating role-playing game quests with gpt language models," *IEEE transactions on games*, vol. 16, no. 1, pp. 127–139, 2022.

[2] G. Smith, R. Anderson, B. Kopleck, Z. Lindblad, L. Scott, A. Wardell, J. Whitehead, and M. Mateas, "Situating quests: Design patterns for quest and level design in role-playing games," in *International Conference on Interactive Digital Storytelling*. Springer, 2011, pp. 326–329.

[3] Y. Wu, Y. Chen, X. Xie, B. Yu, C. Fan, and L. Ma, "Regression testing of massively multiplayer online role-playing games," in *ICSME*. IEEE, 2020, pp. 692–696.

[4] S. McConnell, "Daily build and smoke test," *IEEE software*, vol. 13, no. 4, p. 144, 1996.

[5] C. Cannavacciuolo and L. Mariani, "Smoke testing of cloud systems," in *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2022, pp. 47–57.

[6] J. Hu, M. Zhang, B. Liu, Y. Wu, and Y. Chen, "A language-guided acceleration method for smoke testing of game quests," in *35th IEEE International Symposium on Software Reliability Engineering, ISSRE 2024 - Workshops, Tsukuba, Japan, October 28-31, 2024*. IEEE, 2024, pp. 15–20.

[7] S. Herbold and T. Haar, "Smoke testing for machine learning: Simple tests to discover severe bugs," *Empirical Software Engineering*, vol. 27, no. 2, p. 45, January 2022.

[8] C. Politowski, F. Petrillo, and Y.-G. Guéhéneuc, "A survey of video game testing," in *2021 IEEE/ACM International Conference on Automation of Software Test (AST)*. IEEE, 2021, pp. 90–99.

[9] A. Memon, I. Banerjee, N. Hashmi, and A. Nagarajan, "Dart: a framework for regression testing" nightly/daily builds" of gui applications," in *International Conference on Software Maintenance, 2003. ICSM 2003. Proceedings.* IEEE, 2003, pp. 410–419.

[10] C.-S. Cho, D.-C. Lee, K.-M. Sohn, C.-J. Park, and J.-H. Kang, "Scenario-based approach for blackbox load testing of online game servers," in *2010 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery*, 2010, pp. 259–265.

[11] M. Ostrowski and S. Aroudj, "Automated regression testing within video game development," *GSTF Journal on Computing (JoC)*, vol. 3, no. 2, p. 10, 2013.

[12] S. Iftikhar, M. Z. Iqbal, M. U. Khan, and W. Mahmood, "An automated model based testing approach for platform games," in *2015 ACM/IEEE 18th International Conference on Model Driven Engineering Languages and Systems (MODELS)*, 2015, pp. 426–435.

[13] J. Hernández Bécares, L. Costero Valero, and P. P. Gómez Martín, "An approach to automated videogame beta testing," *Entertainment Computing*, vol. 18, pp. 79–92, 2017. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1875952116300234

[14] J. Bergdahl, C. Gordillo, K. Tollmar, and L. Gisslén, "Augmenting automated game testing with deep reinforcement learning," in *2020 IEEE Conference on Games (CoG)*, 2020, pp. 600–603.

[15] A. Sestini, L. Gisslén, J. Bergdahl, K. Tollmar, and A. D. Bagdanov, "Automated gameplay testing and validation with curiosity-conditioned proximal trajectories," *IEEE Transactions on Games*, vol. 16, no. 1, pp. 113–126, 2024.

[16] C. Gordillo, J. Bergdahl, K. Tollmar, and L. Gisslén, "Improving playtesting coverage via curiosity driven reinforcement learning agents," in *2021 IEEE Conference on Games (CoG)*, 2021, pp. 1–8.

[17] Y. Zheng, X. Xie, T. Su, L. Ma, J. Hao, Z. Meng, Y. Liu, R. Shen, Y. Chen, and C. Fan, "Wuji: Automatic online combat game testing using evolutionary deep reinforcement learning," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019, pp. 772–784.

[18] C. Holmgård, M. C. Green, A. Liapis, and J. Togelius, "Automated playtesting with procedural personas through mcts with evolved heuristics," *IEEE Transactions on Games*, vol. 11, no. 4, pp. 352–362, 2019.

[19] W. Ahlberg, A. Sestini, K. Tollmar, and L. Gisslén, "Generating personas for games with multimodal adversarial imitation learning," in *2023 IEEE Conference on Games (CoG)*, 2023, pp. 1–8.

[20] S. Ariyurek, A. Betin-Can, and E. Surer, "Automated video game testing using synthetic and humanlike agents," *IEEE Transactions on Games*, vol. 13, no. 1, pp. 50–67, 2021.

[21] Y. Duarte, H. C. Mandelli, V. H. S. Durelli, P. A. Nardi, and A. T. Endo, "Exploratory testing for platform video games: strategies and lessons learned," *J. Interact. Syst.*, vol. 15, no. 1, pp. 657–669, 2024.

[22] M. Zhang, X. Zhang, P. Arcaini, and F. Ishikawa, "An investigation of the behaviours of machine learning agents used in the game of go," in *DSA*. IEEE, 2023, pp. 734–742.

[23] P. Spronck, M. Ponsen, I. Sprinkhuizen-Kuyper, and E. Postma, "Adaptive game ai with dynamic scripting," *Machine Learning*, vol. 63, no. 3, pp. 217–248, 2006.

[24] I. Zarembo, "Analysis of artificial intelligence applications for automated testing of video games," in *Proceedings of the 12th International Scientific and Practical Conference. Volume II*, vol. 170, 2019, p. 174.

[25] A. Zook, E. Fruchter, and M. O. Riedl, "Automatic playtesting for game parameter tuning via active learning," *arXiv preprint arXiv:1908.01417*, 2019.

[26] S. Ariyurek, A. Betin-Can, and E. Surer, "Enhancing the monte carlo tree search algorithm for video game testing," *arXiv preprint arXiv:2003.07813*, 2020.

[27] D. Loubos, "Automated testing in virtual worlds," Master's thesis, 2018.

[28] J. Pfau, J. D. Smeddinck, and R. Malaka, "Automated game testing with icarus: Intelligent completion of adventure riddles via unsupervised solving," in *Extended Abstracts Publication of the Annual Symposium on Computer-Human Interaction in Play*, 2017, pp. 153–164.

[29] S. Roohi, J. Takatalo, C. Guckelsberger, and P. Hämäläinen, "Review of intrinsic motivation in simulation-based game testing," in *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, 2018, pp. 1–13.

[30] C. Guerrero-Romero, S. M. Lucas, and D. Perez-Liebana, "Using a team of general ai algorithms to assist game design and testing," in *2018 IEEE Conference on Computational Intelligence and Games (CIG)*. IEEE, 2018, pp. 1–8.

[31] P. Ammanabrolu and M. O. Riedl, "Playing text-adventure games with graph-based deep reinforcement learning," *arXiv preprint arXiv:1812.01628*, 2018.

[32] K. Debnath, R. Omar, N. Abdul Latip, S. Shelyna *et al.*, "A review on graph search algorithms for optimal energy efficient path planning for an unmanned air vehicle," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 15, no. 2, pp. 743–749, 2019.

[33] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine learning*, vol. 47, no. 2-3, pp. 235–256, 2002.

[34] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, and others, "Mastering the game of go without human knowledge," *Nature*, vol. 550, no. 7676, p. 354, 2017.

[35] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov, "Bag of Tricks for Efficient Text Classification," in *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*. Association for Computational Linguistics, Apr. 2017, pp. 427–431.

[36] C. C. Aggarwal and C. Zhai, "A survey of text classification algorithms," in *Mining text data*. Springer, 2012, pp. 163–222.

[37] J. He, J. Chen, X. He, J. Gao, L. Li, L. Deng, and M. Ostendorf, "Deep reinforcement learning with a natural language action space," *arXiv preprint arXiv:1511.04636*, 2015.

[38] D. L. Chen and R. J. Mooney, "Learning to interpret natural language navigation instructions from observations," in *Twenty-Fifth AAAI Conference on Artificial Intelligence*, 2011.

[39] J. Hu, M. Zhang, B. Liu, Y. Wu, and Y. Chen, "A language-guided acceleration method for smoke testing of game quests," in *2024 IEEE 35th International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 2024, pp. 7–12.

[40] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," 2023. [Online]. Available: https://arxiv.org/abs/2305.16291

[41] H. Warpefelt and H. Verhagen, "A model of non-player character believability," *Journal of Gaming & Virtual Worlds*, vol. 9, no. 1, pp. 39–53, 2017.

[42] T. Rist, "Using a large language model to turn explorations of virtual 3d-worlds into interactive narrative experiences," in *2024 IEEE Conference on Games (CoG)*. IEEE, 2024, pp. 1–8.

[43] N. Ranella and M. Eger, "Towards automated video game commentary using generative ai." in *EXAG@ AIIDE*, 2023.

[44] X. You, P. Taveekitworachai, S. Chen, M. Can Gursesli, X. Li, Y. Xia, and R. Thawonmas, "Dungeons, dragons, and emotions: A preliminary study of player sentiment in llm-driven ttrpgs," in *Proceedings of the 19th International Conference on the Foundations of Digital Games*, 2024, pp. 1–4.

[45] T. Wang, M. Honari-Jahromi, S. Katsarou, O. Mikheeva, T. Panagiotakopoulos, S. Asadi, and O. Smirnov, "player2vec: A language modeling approach to understand player behavior in games," *arXiv preprint arXiv:2404.04234*, 2024.

[46] G. Todd, S. Earle, M. U. Nasir, M. C. Green, and J. Togelius, "Level generation through large language models," in *Proceedings of the 18th International Conference on the Foundations of Digital Games*, 2023, pp. 1–8.

[47] E. Decart, Q. McIntyre, S. Campbell, X. Chen, and R. Wachen, "Oasis: A universe in a transformer," *URL: https://oasis-model. github. io*, 2024.

[48] S. Reed, K. Zolna, E. Parisotto, S. G. Colmenarejo, A. Novikov, G. Barth-Maron, M. Gimenez, Y. Sulsky, J. Kay, J. T. Springenberg *et al.*, "A generalist agent," *arXiv preprint arXiv:2205.06175*, 2022.

[49] Z. Wang, S. Cai, A. Liu, Y. Jin, J. Hou, B. Zhang, H. Lin, Z. He, Z. Zheng, Y. Yang *et al.*, "Jarvis-1: Open-world multi-task agents with memory-augmented multimodal language models," *arXiv preprint arXiv:2311.05997*, 2023.

[50] H. Yuan, C. Zhang, H. Wang, F. Xie, P. Cai, H. Dong, and Z. Lu, "Skill reinforcement learning and planning for open-world long-horizon tasks," *arXiv preprint arXiv:2303.16563*, 2023.

[51] K. Nottingham, P. Ammanabrolu, A. Suhr, Y. Choi, H. Hajishirzi, S. Singh, and R. Fox, "Do embodied agents dream of pixelated sheep: Embodied decision making using language guided world modelling," in *International Conference on Machine Learning*. PMLR, 2023, pp. 26 311–26 325.

[52] Y. Du, O. Watkins, Z. Wang, C. Colas, T. Darrell, P. Abbeel, A. Gupta, and J. Andreas, "Guiding pretraining in reinforcement learning with large language models," in *International Conference on Machine Learning*. PMLR, 2023, pp. 8657–8677.

[53] J. Wei, X. Wang, D. Schuurmans, M. Bosma, b. ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35, 2022, pp. 24 824–24 837.

[54] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023, pp. 8634–8652. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/1b44b878bb782e6954cd888628510e90-Paper-Conference.pdf

[55] Z. Li, Y. Wu, L. Ma, X. Xie, Y. Chen, and C. Fan, "Gbgallery: A benchmark and framework for game testing," *Empirical Software Engineering*, vol. 27, no. 6, p. 140, 2022.

[56] J. H. Holland, "Genetic algorithms," *Scientific American*, Jul. 1992.

[57] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[58] J. Cai, J. Li, N. Li, M. Zhang, R. Yang, and K. Tei, "Overcooked plus: A comprehensive cooking scenario testbed for enhancing the evaluation of autonomous planning algorithms," in *Proceedings of the 5th IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*, September 2024.

[59] Ghost Town Games Ltd., "Overcooked!" Team17, 2024, accessed: 2024-08-26. [Online]. Available: https://www.team17.com/games/overcooked/

[60] Mojang Studios, "Minecraft," https://www.minecraft.net, 2009, sandbox video game developed by Mojang Studios.